RESOURCE GUIDE & MATLAB/SIMULINK MODELS

Intelligent Smart Microgrid Energy Management System using ML

PRE-BUILT MATLAB/SIMULINK MODELS

1. Official MathWorks Examples

Microgrid System Examples

Link: https://www.mathworks.com/help/sps/power-grids.html

Available Models:

- **Microgrid Resynchronization Example**
  - Model: `openExample('simscapeelectrical/MicrogridResynchronizationExample')`
  - Features: Battery energy storage, PV solar park, grid connection
  - Modes: Islanded and grid-connected operation
- **Remote Microgrid Design and Operation**
  - Model: `openExample('simscape_shared/DesignOperateControlRemoteMicrogridExample')`
  - Features: Complete remote microgrid with renewables
  - Includes: Droop control, economic analysis
- **IEEE 39-Bus System**
  - Model: `openExample('simscapeelectrical/IEEE39BusSystemExample')`
  - Features: Large-scale power system simulation
  - Use: Grid integration studies

Microgrid Video Series

Link: https://www.mathworks.com/videos/series/microgrid-system-development-and-analysis.html

Topics Covered:

- Distributed power systems concepts
- Microgrid with peak shaving/islanding EMS
- Grid-connected and islanded operation
- Renewable DER integration
- Energy storage systems

2. MATLAB Central File Exchange

Microgrid Hybrid PV/Wind/Battery Management System

Link: https://www.mathworks.com/matlabcentral/fileexchange/60550-microgrid-hybrid-pv-wind-battery-management-system

Features:

- Solar PV with MPPT controller
- Wind turbine system
- Battery management system (BMS)
- Intelligent control integration
- UPFC for fault analysis
- Downloads: 40.6K+ (very popular)
- Rating: 4.5/5 (188 reviews)

Key Components:

```
matlab

% Main components in the model:
- PV Array with Perturb & Observe MPPT
- Wind Turbine with Power Converter
- Battery Storage with Fuzzy Logic Controller
- Grid Connection with Synchronization
- Load Management System
```

Systems-Level Microgrid Simulation

Link: https://www.mathworks.com/matlabcentral/fileexchange/67060-systems-level-microgrid-simulation-from-simple-one-line-diagram

Features:

- Simple one-line diagram approach
- Renewable energy sources (solar)
- Diesel GenSet backup
- Energy storage system (ESS)
- Residential load modeling
- Hardware-in-the-loop ready

3. Microgrid Control Documentation

What Is Microgrid Control? Link: https://www.mathworks.com/discovery/microgrid-control.html

Control Modes Covered:

- Grid synchronization
- Grid forming (islanded mode)
- Grid following
- Droop control implementation
- Power electronics control

Features:

- Multi-fidelity modeling approach
- Real-time implementation support
- HIL testing capabilities

DATA SOURCES FOR THE PROJECT

1. NASA POWER Data Access Viewer

Link: https://power.larc.nasa.gov/

Available Data:

- Solar irradiance (W/m²)
- Temperature (°C)
- Wind speed (m/s)
- Humidity and precipitation
- Historical data (40+ years)
- Global coverage
- API access available

How to Access:

1. Visit https://power.larc.nasa.gov/
2. Select "Data Access Viewer"
3. Choose location (Bharatpur, Rajasthan coordinates)
4. Select parameters: Solar GHI, Temperature, Wind Speed
5. Choose time period: 1-2 years of data
6. Download as CSV format

2. NREL National Solar Radiation Database (NSRDB)

Link: https://nsrdb.nrel.gov/

Features:

- High-resolution solar data
- 30-minute to 5-minute intervals
- Typical Meteorological Year (TMY) data
- Satellite-derived measurements
- Free for research use

Data Types:

- GHI (Global Horizontal Irradiance)
- DNI (Direct Normal Irradiance)
- DHI (Diffuse Horizontal Irradiance)
- Temperature, pressure, wind

3. Open Power System Data

Link: https://open-power-system-data.org/

Available Datasets:

- Load consumption patterns
- Renewable generation profiles
- Market prices
- European focus but useful for patterns

MACHINE LEARNING RESOURCES

1. Python Libraries for ML/DRL

Deep Reinforcement Learning

```bash
# Stable-Baselines3 (recommended)
pip install stable-baselines3[extra]

# Key algorithms available:
# - PPO (Proximal Policy Optimization)
# - DQN (Deep Q-Network)
# - A2C (Advantage Actor-Critic)
# - SAC (Soft Actor-Critic)
# - TD3 (Twin Delayed DDPG)
```

Documentation: https://stable-baselines3.readthedocs.io/

Example Code:

python

```python
from stable_baselines3 import PPO
from stable_baselines3.common.env_util import make_vec_env

# Create microgrid environment
env = make_vec_env('MicrogridEnv-v0', n_envs=4)

# Initialize PPO agent
model = PPO('MlpPolicy', env, verbose=1)

# Train the agent
model.learn(total_timesteps=100000)

# Save the model
model.save("microgrid_ppo")
```

Time Series Forecasting

bash

```bash
# TensorFlow/Keras for LSTM
pip install tensorflow

# PyTorch alternative
pip install torch torchvision

# Scikit-learn for SVR
pip install scikit-learn
```

LSTM Example:

python

```
import tensorflow as tf
from tensorflow import keras

# Define LSTM model for solar forecasting
model = keras.Sequential([
    keras.layers.LSTM(128, return_sequences=True, input_shape=(timesteps, features)),
    keras.layers.Dropout(0.2),
    keras.layers.LSTM(64, return_sequences=False),
    keras.layers.Dropout(0.2),
    keras.layers.Dense(32, activation='relu'),
    keras.layers.Dense(1)  # Output: predicted solar power
])

model.compile(optimizer='adam', loss='mse', metrics=['mae'])
```

2. MATLAB Deep Learning Toolbox

LSTM in MATLAB:

matlab

```
% Create LSTM network for forecasting
numFeatures = 5;  % weather features
numResponses = 1;  % power output
numHiddenUnits = 128;

layers = [ ...
    sequenceInputLayer(numFeatures)
    lstmLayer(numHiddenUnits,'OutputMode','sequence')
    dropoutLayer(0.2)
    lstmLayer(64,'OutputMode','last')
    fullyConnectedLayer(numResponses)
    regressionLayer];

options = trainingOptions('adam', ...
    'MaxEpochs',100, ...
    'GradientThreshold',1, ...
    'InitialLearnRate',0.005, ...
    'LearnRateSchedule','piecewise', ...
    'LearnRateDropPeriod',40, ...
    'LearnRateDropFactor',0.1, ...
    'Verbose',0, ...
    'Plots','training-progress');

net = trainNetwork(XTrain,YTrain,layers,options);
```

3. MATLAB Reinforcement Learning Toolbox

DQN Example:

```matlab
% Create DQN agent for microgrid
obsInfo = rlNumericSpec([8 1]); % 8 state variables
obsInfo.Name = 'observations';

actInfo = rlFiniteSetSpec([1 2 3]); % 3 actions: charge, discharge, hold
actInfo.Name = 'actions';

env = rlFunctionEnv(obsInfo,actInfo,...
    'microgridStepFunction','microgridResetFunction');

% Create DQN agent
agent = rlDQNAgent(obsInfo,actInfo);

% Train agent
trainOpts = rlTrainingOptions(...
    'MaxEpisodes',500, ...
    'MaxStepsPerEpisode',500, ...
    'Verbose',false, ...
    'Plots','training-progress');

trainingStats = train(agent,env,trainOpts);
```

RECOMMENDED TEXTBOOKS & PAPERS

Textbooks

1. **"Microgrid Planning and Design"** by Hassan Farhangi
   - Comprehensive guide to microgrid systems
   - Practical implementation examples
2. **"Deep Learning"** by Ian Goodfellow, Yoshua Bengio, Aaron Courville
   - Free online: https://www.deeplearningbook.org/
   - Chapters 10 (RNN/LSTM) and Chapter 20 (Deep RL)
3. **"Reinforcement Learning: An Introduction"** by Sutton & Barto
   - Free PDF: http://incompleteideas.net/book/the-book.html
   - Essential for understanding DRL

Key Research Papers (Must Read)

2024-2025 Publications:

1. **Singh et al. (2024)**
    - Title: "Machine learning-based energy management and power forecasting"
    - Journal: Scientific Reports, Vol. 14
    - DOI: 10.1038/s41598-024-70336-3
    - Key: SVR for solar/wind forecasting
2. **Spiliopoulos et al. (2025)**
    - Title: "A Smart Microgrid Platform Integrating AI and Deep RL"
    - Journal: Energies, 18(5), 1157
    - DOI: 10.3390/en18051157
    - Key: DRL achieving 23% cost reduction
3. **Eyimaya et al. (2025)**
    - Title: "LSTM Networks for DC Microgrid Optimization"
    - Journal: Energies, 18(14), 3676
    - Key: LSTM vs MLP comparison
4. **Mahmoudabadi et al. (2025)**
    - Title: "Energy management of microgrid coalitions"
    - Journal: AIP Advances, 15(4), 045323
    - Key: Multi-microgrid systems
5. **Silva-Rodriguez et al. (2024)**
    - Title: "LSTM-Based Net Load Forecasting"
    - arXiv preprint
    - Key: Wind and solar equipped microgrids

2023 Publications:

6. **Joshi et al. (2023)**
    - Title: "Survey on AI and ML techniques for microgrid EMS"
    - Journal: IEEE/CAA J. Autom. Sinica, 10(7)
    - DOI: 10.1109/JAS.2023.123657
    - Key: Comprehensive survey of methods

MATLAB-PYTHON INTEGRATION

Method 1: MATLAB Engine API for Python

```bash
# Install MATLAB Engine for Python
cd "matlabroot/extern/engines/python"
python setup.py install
```

Usage Example:

```python
import matlab.engine

# Start MATLAB engine
eng = matlab.engine.start_matlab()

# Run Simulink model
eng.load_system('microgrid_model', nargout=0)
eng.set_param('microgrid_model/SolarPV/Power', 'Value', str(predicted_power), nargout=0)
eng.sim('microgrid_model', nargout=0)

# Get results
results = eng.workspace['simulation_results']
```

Method 2: Python in MATLAB

```matlab
% Call Python from MATLAB
pyenv('Version', 'C:\Python38\python.exe');

% Import Python module
py.importlib.import_module('numpy');

% Use trained ML model
py.pickle.load(py.open('lstm_model.pkl', 'rb'))
```

Method 3: Save/Load Data Files

Python saves predictions:

```python
import scipy.io as sio
sio.savemat('predictions.mat', {'solar_forecast': predictions})
```

MATLAB loads predictions:

```matlab
data = load('predictions.mat');
solar_forecast = data.solar_forecast;
```

PROJECT IMPLEMENTATION ROADMAP

Phase 1: Environment Setup (Week 1)

Software Installation Checklist:

- [ ] MATLAB R2023b with required toolboxes
- [ ] Python 3.8+ with Anaconda
- [ ] Git for version control
- [ ] VS Code or Jupyter for Python development

MATLAB Toolboxes Required:

- [ ] Simulink
- [ ] Simscape Electrical
- [ ] Deep Learning Toolbox
- [ ] Reinforcement Learning Toolbox
- [ ] Optimization Toolbox
- [ ] Statistics and Machine Learning Toolbox

Python Packages:

```bash
pip install tensorflow
pip install torch
pip install stable-baselines3
pip install scikit-learn
pip install pandas numpy matplotlib seaborn
pip install scipy
pip install gym
```

Phase 2: Data Collection (Week 1-2)

Data Collection Script Example:

```python
```

```python
import requests
import pandas as pd

# NASA POWER API example
def get_nasa_data(lat, lon, start_date, end_date):
    base_url = "https://power.larc.nasa.gov/api/temporal/hourly/point"

    params = {
        'parameters': 'ALLSKY_SFC_SW_DWN,T2M,WS10M',  # Solar, Temp, Wind
        'community': 'RE',
        'longitude': lon,
        'latitude': lat,
        'start': start_date,
        'end': end_date,
        'format': 'JSON'
    }

    response = requests.get(base_url, params=params)
    data = response.json()

    # Convert to DataFrame
    df = pd.DataFrame(data['properties']['parameter'])
    return df

# Bharatpur, Rajasthan coordinates
lat = 27.2152
lon = 77.4889

# Get 1 year of data
weather_data = get_nasa_data(lat, lon, '20230101', '20231231')
weather_data.to_csv('bharatpur_weather_2023.csv')
```

Phase 3: ML Model Development Templates

LSTM Solar Forecasting:

```python
```

```python
# lstm_solar_forecast.py
import numpy as np
import pandas as pd
from sklearn.preprocessing import MinMaxScaler
from tensorflow import keras

class SolarForecaster:
    def __init__(self, sequence_length=24):
        self.sequence_length = sequence_length
        self.scaler = MinMaxScaler()
        self.model = None

    def prepare_data(self, data):
        # Normalize data
        scaled_data = self.scaler.fit_transform(data)

        # Create sequences
        X, y = [], []
        for i in range(len(scaled_data) - self.sequence_length):
            X.append(scaled_data[i:i+self.sequence_length])
            y.append(scaled_data[i+self.sequence_length, 0])  # solar power

        return np.array(X), np.array(y)

    def build_model(self, input_shape):
        model = keras.Sequential([
            keras.layers.LSTM(128, return_sequences=True, input_shape=input_shape),
            keras.layers.Dropout(0.2),
            keras.layers.LSTM(64),
            keras.layers.Dropout(0.2),
            keras.layers.Dense(32, activation='relu'),
            keras.layers.Dense(1)
        ])

        model.compile(optimizer='adam', loss='mse', metrics=['mae'])
        self.model = model
        return model

    def train(self, X_train, y_train, epochs=100, batch_size=32):
        history = self.model.fit(
            X_train, y_train,
            epochs=epochs,
            batch_size=batch_size,
            validation_split=0.2,
            verbose=1
        )
```

```
        return history

    def predict(self, X):
        predictions = self.model.predict(X)
        # Inverse transform to get actual values
        predictions = self.scaler.inverse_transform(
            np.concatenate([predictions, np.zeros((len(predictions), self.scaler.n_features_in_-1))], axis=1)
        )[:, 0]
        return predictions
```

DRL Energy Management:

python

```python
# drl_energy_management.py
import gym
from gym import spaces
import numpy as np
from stable_baselines3 import PPO

class MicrogridEnv(gym.Env):
    def __init__(self, solar_forecast, load_forecast, battery_capacity=100):
        super(MicrogridEnv, self).__init__()

        self.solar_forecast = solar_forecast
        self.load_forecast = load_forecast
        self.battery_capacity = battery_capacity
        self.battery_soc = battery_capacity / 2  # Start at 50%

        # State: [solar_power, load_demand, battery_soc, time_of_day, grid_price]
        self.observation_space = spaces.Box(
            low=np.array([0, 0, 0, 0, 0]),
            high=np.array([100, 100, 100, 24, 10]),
            dtype=np.float32
        )

        # Action: [battery_charge_rate] from -1 to 1
        # -1 = max discharge, 0 = no action, 1 = max charge
        self.action_space = spaces.Box(
            low=-1, high=1, shape=(1,), dtype=np.float32
        )

        self.current_step = 0
        self.max_steps = len(solar_forecast)

    def reset(self):
        self.current_step = 0
        self.battery_soc = self.battery_capacity / 2
        return self._get_state()

    def _get_state(self):
        solar = self.solar_forecast[self.current_step]
        load = self.load_forecast[self.current_step]
        soc = self.battery_soc
        time = self.current_step % 24
        price = self._get_grid_price(time)

        return np.array([solar, load, soc, time, price], dtype=np.float32)

    def _get_grid_price(self, hour):
```

```
        # Simple time-of-use pricing
        if 6 <= hour < 10 or 17 <= hour < 22:  # Peak hours
            return 8.0
        elif 10 <= hour < 17:  # Mid-peak
            return 5.0
        else:  # Off-peak
            return 2.0

    def step(self, action):
        solar = self.solar_forecast[self.current_step]
        load = self.load_forecast[self.current_step]

        # Calculate battery action
        battery_action = action[0] * 20  # Max 20 kW charge/discharge

        # Update battery SoC
        self.battery_soc += battery_action
        self.battery_soc = np.clip(self.battery_soc, 0, self.battery_capacity)

        # Calculate grid import/export
        net_power = solar - load - battery_action
        grid_import = max(0, -net_power)
        grid_export = max(0, net_power)

        # Calculate cost
        grid_price = self._get_grid_price(self.current_step % 24)
        cost = grid_import * grid_price - grid_export * grid_price * 0.5

        # Reward = negative cost (we want to minimize cost)
        reward = -cost

        # Penalties for battery violations
        if self.battery_soc < 0.2 * self.battery_capacity:
            reward -= 10  # Penalty for low SoC
        if self.battery_soc > 0.9 * self.battery_capacity:
            reward -= 5  # Penalty for high SoC

        # Move to next step
        self.current_step += 1
        done = self.current_step >= self.max_steps

        return self._get_state(), reward, done, {}

    def render(self):
        pass

# Training script
```

```python
def train_drl_agent():
    # Load forecasts
    solar_forecast = np.random.rand(8760) * 50  # 1 year hourly data
    load_forecast = np.random.rand(8760) * 40

    # Create environment
    env = MicrogridEnv(solar_forecast, load_forecast)

    # Create PPO agent
    model = PPO('MlpPolicy', env, verbose=1, tensorboard_log="./ppo_microgrid_tensorboard/")

    # Train agent
    model.learn(total_timesteps=100000)

    # Save model
    model.save("ppo_microgrid_ems")

    return model
```

SIMULINK MODEL STRUCTURE

Basic Microgrid Model Components

```
Microgrid_Model.slx
│
├── Renewable_Sources/
│   ├── Solar_PV_Array
│   │   ├── Irradiance_Input
│   │   ├── PV_Module (Simscape)
│   │   └── MPPT_Controller
│   │
│   └── Wind_Turbine
│       ├── Wind_Speed_Input
│       ├── Turbine_Model
│       └── Power_Converter
│
├── Energy_Storage/
│   ├── Battery_Pack (Simscape)
│   ├── Battery_Management_System
│   └── Bidirectional_Converter
│
├── Grid_Connection/
│   ├── Grid_Model
│   ├── PCC (Point of Common Coupling)
```

```
│   └── Synchronization_Control
│
├── Load/
│   ├── Residential_Load
│   ├── Commercial_Load
│   └── Load_Profile_Input
│
└── Control_System/
    ├── Central_EMS_Controller
    ├── LSTM_Forecast_Interface (Python)
    ├── DRL_Decision_Interface (Python)
    └── Protection_Logic
```

Simulink Block Recommendations

Solar PV:

```
matlab

% Use Simscape Electrical blocks:
- PV Array block
- DC-DC Converter (Boost)
- MPPT Controller (P&O or InCond)
```

Wind Turbine:

```
matlab

- Wind Turbine block (Simscape)
- AC-DC Rectifier
- DC-DC Converter
```

Battery:

```
matlab

- Battery block (Simscape Electrical)
- Bidirectional DC-DC Converter
- SOC Estimation block
```

Grid Interface:

```
matlab
```

- Three-Phase Source (for grid)
- Inverter (DC-AC)
- Transformer
- Circuit Breaker
- Synchronized breaker control

PERFORMANCE METRICS CALCULATION

Forecasting Accuracy Metrics

python

```python
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
import numpy as np

def calculate_metrics(y_true, y_pred):
    """Calculate comprehensive forecasting metrics"""

    # RMSE
    rmse = np.sqrt(mean_squared_error(y_true, y_pred))

    # MAE
    mae = mean_absolute_error(y_true, y_pred)

    # MAPE
    mape = np.mean(np.abs((y_true - y_pred) / y_true)) * 100

    # R² Score
    r2 = r2_score(y_true, y_pred)

    # Normalized RMSE
    nrmse = rmse / (y_true.max() - y_true.min()) * 100

    metrics = {
        'RMSE': rmse,
        'MAE': mae,
        'MAPE': mape,
        'R2': r2,
        'NRMSE': nrmse
    }

    return metrics

# Usage example
solar_actual = np.array([...])  # actual solar power
solar_predicted = np.array([...])  # LSTM predictions

metrics = calculate_metrics(solar_actual, solar_predicted)
print(f"Solar Forecasting Performance:")
print(f"RMSE: {metrics['RMSE']:.3f} kW")
print(f"MAE: {metrics['MAE']:.3f} kW")
print(f"MAPE: {metrics['MAPE']:.2f}%")
print(f"R²: {metrics['R2']:.4f}")
```

Energy Management Performance

python

```python
def calculate_ems_metrics(simulation_results):
    """Calculate energy management system performance"""

    # Extract data from simulation
    solar_gen = simulation_results['solar_generation']
    wind_gen = simulation_results['wind_generation']
    load = simulation_results['load_demand']
    battery = simulation_results['battery_dispatch']
    grid = simulation_results['grid_import_export']

    # Total renewable generation
    total_renewable = np.sum(solar_gen) + np.sum(wind_gen)

    # Total consumption
    total_load = np.sum(load)

    # Grid import/export
    grid_import = np.sum(np.maximum(0, grid))
    grid_export = np.sum(np.maximum(0, -grid))

    # Renewable utilization rate
    renewable_utilization = (total_renewable - grid_export) / total_renewable * 100

    # Self-sufficiency
    self_sufficiency = ((total_load - grid_import) / total_load) * 100

    # Grid dependency reduction
    grid_dependency = (grid_import / total_load) * 100

    # Battery utilization
    battery_charge = np.sum(np.maximum(0, battery))
    battery_discharge = np.sum(np.maximum(0, -battery))
    battery_cycles = battery_discharge / battery_capacity

    metrics = {
        'Renewable Utilization (%)': renewable_utilization,
        'Self-Sufficiency (%)': self_sufficiency,
        'Grid Dependency (%)': grid_dependency,
        'Battery Cycles': battery_cycles,
        'Total Renewable (kWh)': total_renewable,
        'Grid Import (kWh)': grid_import,
        'Grid Export (kWh)': grid_export
    }

    return metrics
```

Economic Analysis

```python
def calculate_cost_savings(baseline_results, ml_results, electricity_prices):
    """Calculate cost savings with ML-based EMS vs baseline"""

    # Baseline costs
    baseline_cost = np.sum(baseline_results['grid_import'] * electricity_prices['import']) - \
                    np.sum(baseline_results['grid_export'] * electricity_prices['export'])

    # ML-optimized costs
    ml_cost = np.sum(ml_results['grid_import'] * electricity_prices['import']) - \
              np.sum(ml_results['grid_export'] * electricity_prices['export'])

    # Calculate savings
    absolute_savings = baseline_cost - ml_cost
    percentage_savings = (absolute_savings / baseline_cost) * 100

    # Payback period (simplified)
    system_cost = 100000  # Example: $100k system
    annual_savings = absolute_savings * 365 / len(baseline_results)
    payback_period = system_cost / annual_savings

    results = {
        'Baseline Cost ($)': baseline_cost,
        'ML-Optimized Cost ($)': ml_cost,
        'Absolute Savings ($)': absolute_savings,
        'Percentage Savings (%)': percentage_savings,
        'Annual Savings ($)': annual_savings,
        'Payback Period (years)': payback_period
    }

    return results
```

TROUBLESHOOTING GUIDE

Common MATLAB/Simulink Issues

Issue 1: Model runs too slowly

- Solution: Use fixed-step solver instead of variable-step
- Increase solver step size (e.g., 0.001s → 0.01s)
- Simplify power electronic models (use averaged models)

Issue 2: Numerical instability

- Solution: Check parasitic elements (add small resistance/capacitance)
- Use stiff solver (ode15s or ode23s)
- Reduce integration tolerances

Issue 3: Python-MATLAB communication errors

- Solution: Ensure compatible Python version
- Check MATLAB Engine API installation
- Use appropriate data type conversions

Common ML Training Issues

Issue 1: LSTM not converging

- Solution: Normalize input data (use MinMaxScaler)
- Reduce learning rate
- Add more training data
- Simplify model architecture

Issue 2: DRL agent not learning

- Solution: Check reward function (should provide clear signal)
- Increase exploration (higher epsilon)
- Normalize observations
- Try different hyperparameters

Issue 3: Overfitting

- Solution: Add dropout layers
- Use early stopping
- Increase training data
- Apply regularization

ADDITIONAL RESOURCES

Online Courses

1. **MATLAB Onramp**
    - Link: https://www.mathworks.com/learn/tutorials/matlab-onramp.html

- Free introduction to MATLAB

2. **Simulink Onramp**
    - Link: https://www.mathworks.com/learn/tutorials/simulink-onramp.html
    - Learn Simulink basics
3. **Deep Learning Specialization (Coursera)**
    - Instructor: Andrew Ng
    - Covers RNN, LSTM, and more
4. **Reinforcement Learning Specialization (Coursera)**
    - University of Alberta
    - Comprehensive RL course

YouTube Channels

1. **MATLAB** - Official channel with tutorials
2. **Sentdex** - Python and ML tutorials
3. **Two Minute Papers** - Latest AI research explained
4. **Steve Brunton** - Control systems and ML

Forums & Communities

1. **MATLAB Central** - Q&A forum
2. **Stack Overflow** - Programming questions
3. **Reddit r/MachineLearning** - ML discussions
4. **Reddit r/reinforcementlearning** - RL specific

PROJECT DELIVERABLES CHECKLIST

Code & Models

- [ ] MATLAB/Simulink microgrid model (.slx file)
- [ ] Python ML models (LSTM, SVR, DRL)
- [ ] Training scripts and saved model weights
- [ ] Data preprocessing scripts
- [ ] Results visualization scripts

Documentation

- [ ] Technical report (30-50 pages)
- [ ] User guide for Simulink model

- [ ] README with setup instructions
- [ ] Code documentation and comments
- [ ] Performance analysis report

Presentation Materials

- [ ] PowerPoint presentation (15-20 slides)
- [ ] Demo video (5-10 minutes)
- [ ] Poster (if required)

Data

- [ ] Weather dataset (processed)
- [ ] Load profiles
- [ ] Training/validation/test splits
- [ ] Results data (CSV/Excel)

This comprehensive resource guide provides all necessary tools, models, and references for successful project implementation!